

Naval Convoy Protection

Architecture, Layers, and Contracts

Three trust zones - off-chain drones, a sovereign Cairo L2, and an Ethereum-compatible settlement L1 - composed so that on-chain settlement reveals only sealed commitments and verified affidavits, never the underlying drone telemetry.

This document accompanies the thesis defence. It covers the three trust zones, the customisations applied on top of vanilla StarkWare and Madara stacks, every contract used at every layer with the rationale for using it, the current security posture (sound, complete, hiding+binding commitments, non-ZK proof transcript), and the gaps itemised as Future Work.

1. System Overview

Two swarms of five drones each (Alpha and Bravo) sweep a SAFE_AREA ahead of a naval convoy. Each drone must prove to the convoy commander that its assigned strip of the area is safe before the convoy is allowed to advance through it. Proving is done with zk-STARKs over the Cairo CPU; the proof is verified on-chain by StarkWare's GpsStatementVerifier embedded in our Geth L1.

The convoy advance order is gated on a dual-mission SAFE flag: both Alpha and Bravo missions must aggregate five SAFE drone proofs each before the commander key can post the advance order. The cryptographic gate does not auto-fire - the commander always triggers explicitly (Pattern B). The commander key is immutable by design: fail-closed semantics, no rotation path on-chain.

1.1 The three trust zones

L3 - Off-chain (drone)	Runs ArduPilot/Pixhawk flight software, sensors, the predicate Cairo program (safe_area_verify.cairo), and the Stone prover. Holds the full witness: raw cell positions, threat probabilities, timestamps, and the secret nonce. Nothing outside this zone ever sees the raw cells.
L2 - Madara (Cairo rollup)	Sovereign rollup we operate. Mirrors the convoy state, runs the Cairo Verifier for inner-proof recursion, derives each drone's strip from the mission spec, and Pedersen-chains the five commitments into a single aggregate when a swarm completes. Audit-capable because we own the sequencer.
L1 - Geth (Ethereum-compatible)	Settlement layer. Holds the canonical mission specs, the per-drone verdicts, the mission-level SAFE flag, and the convoy advance ledger. Delegates STARK math to StarkWare's audited mainnet verifier stack vendored under contracts/lib/starkware-mainnet/. Sees only commitments and proofs, never raw cells.

2. Privacy Posture - The Sealed Envelope Analogy

The system is sound and complete (STARK guarantees), with an application-level hiding+binding commitment over the drone's cells. Strict transcript-level zero-knowledge requires Sec.5 trace-blinding (Ben-Sasson 2018) which no production Cairo prover currently ships by default - that gap is itemised as Future Work.

2.1 The cast

Patrol log	The drone's witness - raw cells (x,y), per-cell p_contact and timestamps, the nonce. Lives only at L3.
Sealed envelope	The Pedersen-chain commitment H. Binding (drone cannot later open to a different log) and hiding (the secret nonce makes the seal indistinguishable from any other patrol's seal).
Notarised affidavit	The Stone STARK proof. A small cryptographic artefact that certifies the patrol log satisfies the four SAFE_AREA rules - strip containment, coverage $\geq 95\%$, $p_contact < 0.7$, time window ≤ 6 min - without disclosing the log itself.
HQ guard	GpsStatementVerifier on L1. Checks the affidavit is mathematically authentic. Does not re-execute the Cairo program; runs $O(\log^2 n)$ field operations against the proof.

Public bulletin board	Geth L1 - Registry, Verifier, CommandLog. Anyone observing the chain sees only receipts (strip bounds, verdict, mission id, drone id), sealed envelopes (H values), affidavits (proof bytes), master seals (aggregate H), and advance orders.
Interior vault	Madara L2 - ConvoyProtocol. We own this rollup, so we can let an authorised auditor open envelopes here without anything leaving the vault. The lever exists architecturally; the audit endpoint is Future Work.

2.2 What is visible at each boundary

L3 - drone	Full patrol log, nonce, raw trace, all eight public output felts.
L3 -> L2 payload	(mission_id, drone_id, H, verdict_bool, proof). Strip bounds + verdict are inside the proof's public memory. Raw cells stay at L3.
L2 -> L1 payload	Same content wrapped as an 11-field SafeProofInputs struct + the EVM-adapted proof arrays. Raw cells never cross.
L1 public state	Mission specs, per-drone records (mission id, drone idx, strip bounds, verdict, H, nSteps, timestamp, blockNumber), mission aggregate H, advance orders. No telemetry.

2.3 The four formal properties

- Soundness [OK] - FRI + Fiat-Shamir at ~60 bits with our prover params; mainnet uses ~80.
- Completeness [OK] - the Cairo program always terminates; verdict=0 is a valid proof of NOT-SAFE.
- Binding [OK] - Pedersen commitment binding under DLP on the STARK curve.
- Hiding (commitment) [OK] - secret nonce makes H computationally hiding.
- Zero-knowledge (proof transcript) [X] - Stone and Stwo do not blind the trace by default. Closes via Sec.5 - Future Work.

3. Contracts by Layer

3.1 L3 - Off-chain (drone)

safe_area_verify.cairo	Cairo 0 program. Encodes the four SAFE_AREA predicates and emits an 8-felt public output: (mission_id, drone_id, x_start, x_end, y_start, y_end, verdict_bool, H). The secret nonce is introduced via a program_input hint; the Pedersen chain over cells + nonce is computed inside the AIR so the commitment is bound to the same witness the predicates checked. Built in Cairo 0 because (a) Stone consumes that flavour, (b) cairo-vm 3.2.0 (the Rust VM Stwo would force us onto) rejects arbitrary program_input hints - confirmed by the Stwo spike.
Stone prover binary	Not a contract, but the entity whose signature L1 will accept. Produces the proof in StarkWare's annotated format, ready for stark-evm-adapter conversion.

3.2 L2 - Madara (Cairo 1, sovereign rollup)

ConvoyProtocol	cairo/convoy_protocol/src/lib.cairo - the L2 state mirror. Responsibilities: (1) #[l1_handler] open_mission bridges mission specs in from L1; (2) submit_commitment(mid, did, H, verdict, proof) accepts per-drone proofs, derives the expected strip from (zone, drone_id, strip_width) and asserts the proof's public bounds match, runs the Cairo Verifier on the inner proof, stores the per-drone H; (3) aggregate_commitment(mid) Pedersen-chains the five H values and send_message_to_l1_syscall when all drones SAFE. Lives on L2 because Cairo native Pedersen is cheap and recursive proof verification is feasible in Cairo but not in Solidity.
-----------------------	--

3.3 L1 - Geth - our application contracts

Registry.sol	Single source of truth for what counts as SAFE for this convoy. Holds MissionSpec (zone dims, nDrones, stripWidth, coverageMin, pMin, timeWindow, areaHash), per-drone verdicts, the per-mission safeCount, the mission-level missionSafe flag, and the aggregate H_swarm. Commander (D's separate signing key, not the validator key) deploys specs; Verifier writes verdicts; CommandLog reads isDualSafe. Splitting state from verification lets future auditing / payout / replay-prevention contracts read the same canonical record.
Verifier.sol	Stage B of the two-stage verification model. Receives ONLY the 11-field SafeProofInputs tuple - never the proof bytes. Owns per-mission relay whitelisting (relayOf[missionId]: 1 -> ship F, 2 -> ship B), derives the expected strip from (spec, droneIndex) and rejects mismatches, asserts starkVerifier.isValid(factHash) is true (a cheap state read against GpsStatementVerifier's FactRegistry, populated by path-a-runner in Stage A), stores a ProofRecord per drone, increments safeCount, and fires Registry.setMissionSafe(mid, aggH) when the swarm completes. Per-drone calldata is ~250 bytes; the gas cost of STARK verification was already paid in Stage A.
CommandLog.sol	The advance-order ledger. The only contract bound to the commander modifier. Calls Registry.isDualSafe(alphaMid, bravoMid) and appends an AdvanceRecord. Separating this from Verifier is what makes the protocol Pattern B: D explicitly triggers the advance, the cryptographic gate does not auto-fire. Fail-closed: commander key set at deploy time, no rotation path.
StarknetCoreStub.sol	Madara's L1<->L2 bridge stub. Madara checks for it at startup. Provides the consumeMessageFromL2 + sendMessageToL2 surface ConvoyProtocol's L1 handlers need. We use a stub rather than the full StarkNet core because we operate the Madara node ourselves and only need the message-passing entry points.
IS StarkVerifier.sol	Minimal interface to GpsStatementVerifier - exposes verifyProofAndRegister(...) and isValid(bytes32). Lets us swap between the real verifier and the mock without recompiling.
MockStarkVerifier.sol	Dev-only path. Accepts any proof unconditionally and registers the fact. Used by fast unit tests that do not carry real proof bytes; production / thesis-defence deployments wire the real GpsStatementVerifier instead.
StarkexBarrel.sol	Forge-compilation stub. Just imports the StarkWare mainnet Solidity files so foundry compiles them, letting DeployStarkVerifier deploy each by artifact name via vm.deployCode(File.sol:Contract). Not deployed itself.

3.4 L1 - Geth - vendored StarkWare mainnet stack

These contracts live under `contracts/lib/starkware-mainnet/` and are Sourcify-verified mainnet source (byte-for-byte modulo metadata) of the contracts deployed at the well-known StarkWare addresses. We deploy our own copies via `DeployStarkVerifier.s.sol` so the system is self-contained on our Geth, but the source is unchanged - we are not reimplementing audited cryptographic code.

GpsStatementVerifier	Public entry point. ConvoyProtocol's Verifier calls <code>verifyProofAndRegister</code> on this. Mainnet address <code>0x9fb7F48dCB26b7bFA4e580b2dEf637B13751942</code> .
CpuFriessVerifier (layout 6)	Cairo CPU AIR verifier for the 'starknet' layout (7 builtins: output, pedersen, range_check, ecdsa, bitwise, ec_op, poseidon). Matches stark-evm-adapter's <code>cairoVerifierId=6</code> default.
CairoBootloaderProgram	Wraps task PIEs in the keccak-stripped 7-builtin bootloader so <code>GpsStatementVerifier</code> accepts them at <code>registerPublicMemoryMainPage</code> .
MemoryPageFactRegistry	Registers the public memory pages of a Cairo execution.
MerkleStatementContract	Verifies trace Merkle commitments before the main proof call.
FriStatementContract	Verifies FRI layer commitments before the main proof call.
CpuOods + periodic columns	Out-of-domain sampling evaluator and the 10 leaf polynomial contracts (<code>CpuConstraintPoly</code> , <code>PedersenX/Y</code> , <code>EcdsaX/Y</code> , four Poseidon round-key columns) - the math backbone of the CPU AIR. Deployed as individual contracts because their bytecode is near the EIP-170 24 KB limit.

4. Customisations Over Vanilla Stacks

4.1 On top of StarkWare's mainnet verifier

- Bootloader hash regenerated locally. We strip the keccak builtin from cairo-bootloader's 7-builtin bootloader (`scripts/strip-keccak-from-bootloader.py`) and compute `SIMPLE_BOOTLOADER_HASH` locally; the deployed `GpsStatementVerifier` is constructed with that hash so the stack accepts our locally-built proofs.
- `numSecurityBits` relaxed from 80 to 60 in `DeployStarkVerifier.s.sol` so our prover params (`n_queries=16`, `log_n_cosets=2`, `proof_of_work_bits=30` -> 62 bits) clear the gate. Documented as a knob to tighten before production.
- `cairoVerifierId` fixed to 6 at the array slot - matches stark-evm-adapter's hardcoded default.

4.2 On top of Madara

- Sovereign mode - we run the sequencer ourselves so the L2 audit lever (open commitments inside the rollup without leaking to L1) is available to us.
- L1<->L2 messaging wired through `StarknetCoreStub` - the smallest shim that satisfies Madara's startup contract check.
- `ConvoyProtocol` does inner-proof verification via the Cairo Verifier - the recursive composition with Stone at L1 is what makes the per-drone cost cheap enough at L1: L1 verifies one outer proof attesting to L2 verifying five inner proofs per mission.

4.3 On top of vanilla zk-STARK posture

- Application-level hiding via Pedersen-chain commitment with a secret nonce. Even without Sec.5 trace-blinding, the on-chain H is computationally hiding so the on-chain observer cannot reconstruct the cells.

- Verdict-bool model: the Cairo program ALWAYS produces a valid proof. `verdict_bool` is 1 for SAFE, 0 for NOT-SAFE. This means the negative case is also auditable - you can prove your strip was found unsafe without falling off the verification path.
- Strip-bounds gate at the Verifier - L1 derives the expected (`x_start`, `x_end`, `y_start`, `y_end`) from the spec and the drone index and asserts the proof's public bounds match. Prevents a drone from sweeping a different strip than the one assigned to it.
- Per-mission relay whitelisting: ship F is the only relay allowed to submit Alpha proofs; ship B for Bravo. Mirror enforced in the Python submitter (`submit_proof_l1.py`) so wrong-key submissions fail loudly client-side.

5. Pipeline Status

5.1 What works end-to-end (code level, wired)

- L3 -> L2: drone runs `safe_area_verify`, Stone proof + 8-felt output. `submit_commitment` accepts on Madara, derives strip, runs Cairo Verifier, stores commitment.
- L2 aggregation: five drones complete -> ConvoyProtocol Pedersen-chains the H values and `send_message_to_l1_syscall` fires.
- L2 -> L1: Verifier.`registerSafeProof` accepts `SafeProofInputs` (11 fields), derives expected strip, asserts match, calls `GpsStatementVerifier`, stores `ProofRecord`, increments `safeCount`, fires `setMissionSafe` when count hits `nDrones`.
- L1 advance: `CommandLog.advance(alpha, bravo, speed)` gated by `Registry.isDualSafe`; commander key immutable.
- `entrypoint.sh` watcher supports per-drone runs and ALL_SAFE / ALL_UNSAFE / ALL_MIXED scenario sweeps.

5.2 The Stage A / Stage B refactor

STARK math and application bookkeeping are now cleanly separated:

Stage A - path-a-runner (Rust binary, runs against StarkWare contracts)

- Phase 1: `MerkleStatementContract.verify()` -- trace decommitments
- Phase 2: `FriStatementContract.verify()` -- FRI layer commits
- Phase 3: `MemoryPageFactRegistry.registerContinuousMemoryPage()`
- Phase 4: `GpsStatementVerifier.verifyProofAndRegister()`
- Effect: `GpsStatementVerifier.isValid(factHash)` returns true.
- Contract addresses env-driven so re-deploys do not rebuild the binary.

Stage B - Verifier.sol (the convoy application contract)

- Receives ONLY the 11-field `SafeProofInputs` tuple (no proof bytes).
- Reads `starkVerifier.isValid(factHash)` -- cheap state lookup.
- Reverts immediately if Stage A never ran for this (program, output).
- Runs the strip-bounds gate, writes `Registry.verdict`, aggregates.

MockStarkVerifier

- Moved out of `contracts/src/` into `contracts/test/`. NOT compiled into production deployments.
- Exposes `setFactValid(factHash, true)` so unit tests can simulate Stage A without running path-a-runner.

- DeployL1.s.sol now refuses to deploy without STARK_VERIFIER_ADDR. Fail loud.

Gas + calldata wins

- Per-drone tx to Verifier.sol shrank from ~800 KB (four EVM-adapted arrays) to ~250 bytes (11 fields).
- STARK math runs ONCE in Stage A, never duplicated in Stage B.

5.3 Mission Data Simulator

scripts/generate-mission.py emits the per-drone program-input JSONs that drive safe_area_verify.cairo. Six scenarios are supported; the first three exercise the predicate paths inside the AIR and the last three exercise the operational handling of drone failures - the convoy-must-hold path.

5.3.1 Predicate scenarios

--scenario both-safe	Every drone in both swarms sweeps a full strip with p_contact < 0.7 and elapsed <= 6 min. All 10 verdict_bool=1; safeCount hits nDrones on both missions; missionSafe flips; CommandLog.advance succeeds.
--scenario both-unsafe	alpha[3] sweeps only the first half of its strip (coverage_permille = 500 < 950) - verdict=0. bravo[3] has one cell with p_contact = 8500 > P_MIN - verdict=0. Both missions stay under nDrones; CommandLog.advance reverts.
--scenario mixed	Alpha all SAFE; bravo[4] high-contact. Single-flank failure: alpha completes but bravo does not. Tests that isDualSafe requires BOTH missions, not either-or.

5.3.2 Dropout (operational stress) scenarios

Models the cases where a drone physically cannot complete its sortie. The system must hold the convoy advance gracefully, with no crash, no off-by-one in safeCount, and a clear operational signal that a sector is uncovered.

--scenario alpha-dropout-vanish	alpha[3] is reported lost. No telemetry was recovered, so no input JSON is written and no proof is generated. entrypt.sh's ALL_VANISH trigger logs 'no telemetry for alpha3' and continues. safeCount caps at 4. missionSafe stays false. Convoy holds.
--scenario alpha-dropout-midflight	alpha[3] sweeps approximately 40% of its strip then vanishes - partial telemetry is recovered by the relay. A Cairo proof CAN still be generated against that partial cell list, verdict_bool = 0 because coverage fails the >= 95% gate. The proof lands on L1 with verdict=0; safeCount stays at 4 (the verdict=0 entry does not increment it). Convoy holds. Same on-chain outcome as vanish but with auditable partial-telemetry evidence.
--scenario dual-dropout	Worst case. alpha[3] vanishes entirely; bravo[4] disappears mid-flight. Neither mission completes. Both missions hold. The vanish_manifest.json names alpha[3] explicitly; bravo[4]'s partial proof lands with verdict=0.

5.3.3 vanish_manifest.json

Every scenario emits a vanish_manifest.json alongside the input files. Drones tagged 'vanish' have one entry each carrying (swarm, mission_id, drone_id, strip bounds, reason). entrypt.sh announces the manifest at the top of an ALL_* run so the operator sees the sector blackouts before proving starts. Empty list when no drones vanished, so consumers can rely on the file existing.

```
{
  "scenario": "dual-dropout",
  "summary": "alpha[3] vanishes, bravo[4] midflight...",
  "vanished": [
```

```
{
  "swarm":      "alpha",
  "mission_id": 1,
  "drone_id":   3,
  "strip_x_start": 6, "strip_x_end": 9,
  "strip_y_start": 0, "strip_y_end": 8,
  "reason":     "vanished (no telemetry recovered)"
}
]
```

5.3.4 End-to-end scenario runner

scripts/run-scenario.sh drives one scenario from generation through proving through L1 verification and asserts the on-chain outcome (Registry.isMissionSafe, Registry.safeCount, CommandLog.advance success/revert) matches what the scenario predicts. scripts/run-all-scenarios.sh runs all six and emits a pass/fail summary. Both shell scripts depend only on cast (foundry), docker, and python3 + the convoy repo. Required env: GETH_RPC_URL, REGISTRY_ADDR, COMMANDLOG_ADDR, COMMANDER_PK, STARK_VERIFIER_ADDR + the four StarkWare contract addresses for path-a-runner.

5.3.5 Trigger-loop integration

entrypoint.sh's prove_trigger watcher accepts shortcuts that map to the scenario directories one-for-one:

```
ALL or ALL_SAFE      -> missions/both-safe/
ALL_UNSAFE           -> missions/both-unsafe/
ALL_MIXED            -> missions/mixed/
ALL_VANISH           -> missions/alpha-dropout-vanish/
ALL_MIDFLIGHT        -> missions/alpha-dropout-midflight/
ALL_DUAL_DROPOUT     -> missions/dual-dropout/

# Per-drone override (drive any input file):
input=/proofs/missions/dual-dropout/alpha1_input.json tag=alpha1
```

6. Future Work

Headline item is strict zero-knowledge. The math is settled - hexens.io/blog/zk-in-starks describes the two complementary masking layers (witness masking via trace polynomials multiplied by the vanishing polynomial of the trace domain, and DEEP composition polynomial masking that injects fresh randomness into the FRI input). What is NOT settled is the engineering: the Stone Prover README does not mention zero-knowledge, blinding, masking, or witness randomisation, and ships without those techniques enabled. Stwo's README is explicit that it is non-ZK by default. Closing the gap therefore means forking a production Cairo prover and bolting on Sec.5-style blinding, or waiting for upstream. Documented as the primary Future Work item because the architecture above the prover is already ZK-ready - dropping in a ZK-Stone binary would not require any contract change.

Stone fork w/ Sec.5 trace-blinding	Witness-polynomial masking (add a random low-degree poly times the vanishing poly of the trace domain) + DEEP composition masking. Removes the residual transcript-leakage gap and gives full zero-knowledge end-to-end. Prover-binary change only - every contract above stays byte-identical.
---	---

Cairo 1 rewrite + Stwo migration	Stwo's cairo-vm 3.2.0 rejects the program_input hints we rely on; safe_area_verify must be re-expressed in Cairo 1 with whitelisted hints only. Pairs naturally with the trace-blinding effort since both happen on the prover side.
Poseidon-chain commitments	Replace Pedersen-chain with Poseidon for speed inside the AIR (Poseidon is a native Cairo builtin; Pedersen is not on EVM) and to let L1 reproduce the chain on-chain if Verifier-side opening becomes desirable.
Threshold/MPC nonce generation	Distribute nonce generation across the 5 drones of a swarm (e.g. DKG) so opening requires k-of-n cooperation. Mitigates single-drone coercion.
ArduPilot SITL -> Pixhawk HITL	Hardware-in-the-loop integration so proof submission is gated by attested-enclave software on the physical drones.
MQTT plane between drones and relay ships	Operational layer - lossy radio link, replay-prevention via the L1 block number written into ConvoyAdvance events.

7. Appendix - Canonical Data Shapes

7.1 Cairo program public output (8 felts, in order)

```
[ mission_id, drone_id, x_start, x_end, y_start, y_end,
  verdict_bool, H ]

mission_id  in {1 (Alpha), 2 (Bravo)}
drone_id    in [1, spec.nDrones]   (1..5 in our config)
(x_start..y_end) strip bounds in zone units
verdict_bool in {0, 1}              1 => SAFE
H            Pedersen-chain over cells + nonce
```

7.2 L1 SafeProofInputs (Verifier.sol, 11 fields)

```
struct SafeProofInputs {
    bytes32 programHash; // keccak of safe_area_verify.cairo
    bytes32 outputHash;  // keccak of the 8-felt output sequence
    uint256 missionId;
    uint8  droneIndex;   // 1..spec.nDrones
    uint32 stripXStart;
    uint32 stripXEnd;
    uint32 stripYStart;
    uint32 stripYEnd;
    uint8  verdictBool;  // 0 or 1
    bytes32 commitment;  // H_i (hiding Pedersen-chain)
    uint256 nSteps;
}
```

7.3 Mission spec (Registry.sol)

```
struct MissionSpec {
    bytes32 areaHash; // Poseidon hash of polygon vertices
    uint32  zoneX;
    uint32  zoneY;
    uint32  zoneW;    // 15 (Alpha) or 20 (Bravo)
    uint32  zoneH;    // 8 (both)
    uint8   nDrones;  // 5
}
```

```
uint32 stripWidth;    // zoneW / nDrones, exact
uint16 coverageMin;   // permille; 950 = 95%
uint16 pMin;          // bp;      7000 = 0.70
uint64 timeWindow;    // seconds; 360 = 6 min
}
```

7.4 Strip derivation formula

```
x_start = zoneX + (droneIndex - 1) * stripWidth
x_end   = x_start + stripWidth
y_start = zoneY
y_end   = zoneY + zoneH
```

Asserted both in `safe_area_verify.cairo` (witness check) AND in `Verifier.sol` (public-input check) - the two checks must agree for `registerSafeProof` to succeed.